

AWS Certified Generative AI Developer — Professional

The Use Case

- Multinational financial institution, operations in 30+ countries
- **Challenges:** decades-old legacy systems holding critical financial data but **lacking modern APIs**; regulatory requirements differing by jurisdiction; high security standards; consistent AI across web, mobile and branch; human oversight of regulated communications; continuous delivery without disrupting critical operations

Note on scope: many services here are not GenAI-specific — Transit Gateway, Direct Connect, Network Firewall, Control Tower, WAF, RAM. This is classic AWS networking and governance applied to a GenAI workload. Prior Solutions Architect knowledge makes this task much easier.

The central architecture: GenAI Gateway

Every application -> GenAI Gateway -> Bedrock / SageMaker

One central layer through which all enterprise FM access flows, so access control, monitoring, cost tracking and governance live in a single place. Covered in full in section 5.

1. Enterprise Connectivity

The problem: the legacy system speaks SOAP, or fixed-width files, or a message queue. It has no modern API — and it cannot be replaced, because it holds critical financial data and it works. So a translation layer goes in front of it.

API Gateway in front of legacy systems

Legacy XML -> mapping template -> JSON -> Bedrock
Bedrock JSON -> mapping template -> XML/SOAP -> Legacy

- **Request/response mappings** mean the legacy system never changes — it keeps sending exactly what it always sent
- **Custom domain mappings** maintain consistent branding (e.g. `ai.bank.com`)

Endpoint type	Use when
Regional	Users are in that region — lower latency
Edge-optimized	Global distribution via CloudFront

Lambda adapters — protocol conversion

Same adapter pattern as Task 1.2, but here it bridges *protocols* rather than models: SOAP → REST, fixed-width → JSON, EBCDIC → UTF-8.

Concurrency setting	What it does
Reserved concurrency	A cap — this function will never exceed N concurrent executions, so it cannot consume the whole account's capacity
Provisioned concurrency	Pre-warmed instances — no cold starts , giving predictable latency under high volume from legacy systems

EventBridge — loose coupling

```
{
  "source": ["banking.transactions"],
  "detail": {
    "amount": [{"numeric": [ ">", 10000 ]}],
    "type": ["international"]
  }
}
```

- **Event pattern matching** means only events that genuinely need FM processing are routed — the rest are ignored
- **Dead-letter queues** catch failed event processing. In finance, events are audit-relevant and **must not be silently dropped**.

Step Functions — two state types

State	Purpose
Parallel	Different branches concurrently (as in Task 1.3)
Map	The same work across many items — batch operations, e.g. one workflow applied to 1,000 transactions

Plus timeout configuration and error handling strategies.

Data synchronisation — Glue vs AppFlow

Service	Suited to	Key feature
AWS Glue	S3, databases, large ETL jobs	Job bookmarks — Glue remembers how far it processed, so the next run handles only new records. No redundant reprocessing.
Amazon AppFlow	SaaS applications (Salesforce, ServiceNow, Slack)	Pre-built connectors, bidirectional flows, filtering and mapping so only relevant data syncs, encryption in transit and at rest

Job bookmarks are the same incremental-sync idea as Task 1.4, built into the service.

Amazon MQ — legacy messaging

- Older enterprise systems speak **JMS, AMQP, MQTT** — not SQS. Amazon MQ is managed ActiveMQ/RabbitMQ.
- **Active/standby deployment** for high availability; security groups and network ACLs restrict access

Discrimination: if the legacy system already speaks JMS/AMQP → **Amazon MQ**. If you are building something new → **SQS** (cheaper, simpler). MQ exists so the legacy side doesn't have to change.

2. Integrated AI Capabilities

Section 1 connected legacy systems. This section enhances *existing applications* with GenAI.

API Gateway usage plans

```
Team A: 1000 requests/day, 10 req/sec
Team B: 5000 requests/day, 50 req/sec
Team C: 100 requests/day, 2 req/sec
```

API keys plus usage plans give **throttling** (no resource exhaustion from one team's bug) and **monitoring** (per-team consumption and cost allocation). This is a core GenAI Gateway capability.

Webhook handlers and idempotency

```
Webhook: "loan application approved"
-> FM call -> email sent
Webhook arrives again (duplicate)
-> FM call -> email sent AGAIN    <- wrong, and paid for twice
```

External systems deliver **at-least-once**. Duplicates *will* arrive. Idempotency is your responsibility: check the event ID against DynamoDB (with TTL), and if it has been seen, return the stored result instead of reprocessing.

Combine with retry and exponential backoff for transient failures.

EventBridge input transformers

```
System A: {"customerId": "123", "amt": 5000}
System B: {"customer_id": "123", "amount": 5000.00}
System C: {"cust": {"id": "123"}, "value": "5000"}
          | input transformer
          -> {"customer_id": "123", "amount": 5000}
```

Normalisation happens **on the event bus**, so no normalisation code is needed in Lambda and the model always receives a consistent structure.

AppSync resolvers with VTL

```
GraphQL request -> VTL mapping template -> model input
model output    -> VTL mapping template -> GraphQL response
```

Direct Lambda integration for real-time inference. **VTL (Apache Velocity Template Language)** performs the transformation declaratively, without writing code.

SQS FIFO and message group IDs

```
1. "check account balance"
2. "execute transfer"
3. "send confirmation"      <- order matters
```

```
Customer A's messages -> group "customer-A"
Customer B's messages -> group "customer-B"
```

Order is preserved within a group; different groups process in parallel. Putting everything in one group would serialise the whole system. Deriving the group ID from **business context** (customer ID, account ID) gives ordering and throughput together.

Amplify DataStore — offline-first clients

- **Selective sync** — only relevant data reaches the device
- **Conflict resolution** — the user edited offline while the server also changed; strategies include last-write-wins, custom logic, optimistic concurrency
- Useful for branch staff working with poor connectivity, caching FM outputs locally

Response mappings and multi-level caching

- **API Gateway response mappings** transform FM output back into legacy formats — content type conversion and structural transformation
- **Caching at three levels:** API Gateway (whole response), application layer (ElastiCache), database (query results) — each needing an invalidation mechanism

Client libraries / SDKs

Rather than every team writing its own Bedrock-calling code, publish a shared library with error handling, retry logic, observability and a consistent interface. Best practice lives in one place, and one team's bug doesn't propagate.

3. Secure Access Frameworks

The densest section — seven services, each a distinct layer.

Identity federation

Service	For
IAM Identity Center	Workforce — employees, internal applications
Amazon Cognito	Customers — external users

The bank already has 50,000 identities in Active Directory; federation means never recreating them in AWS.

Cognito pre-token generation Lambda trigger

```
base token: user_id, email
| pre-token Lambda
enriched:   user_id, email,
           department: "wealth_management",
           clearance:  "confidential",
           region:     "EU"
```

The Lambda pulls enterprise role information (from AD or a database) and injects it into the JWT, so authorisation decisions can be made on those claims. Same mechanism as Marie's `custom:jurisdiction` in Task 1.1, extended to enterprise roles. **Custom authentication flows** handle legacy auth systems that don't speak modern protocols.

IAM condition keys and permission boundaries

```
"Condition": {
  "IpAddress":      {"aws:SourceIp": "10.0.0.0/8"},
  "DateGreaterThan": {"aws:CurrentTime": "09:00Z"},
  "DateLessThan":   {"aws:CurrentTime": "18:00Z"}
}
```

Access only from the corporate network, only during business hours. Conditions can also key off request parameters.

Permission boundaries set a ceiling on delegated administration: a team lead can grant access to their team, but never beyond the boundary — they cannot accidentally create an admin.

Verified Permissions and Cedar

```
permit(principal, action, resource)
when {
  principal.department == "wealth_management" &&
  resource.classification <= "confidential" &&
  context.business_context == "client_meeting"
};
```

Service	Answers
IAM	Can you call Bedrock at all?
Verified Permissions	Can you call it on <i>this document</i>?

Cedar policies authorise based on **data classification level, user department and business context**. **Policy templates** keep governance consistent across many similar policies.

KMS multi-Region keys

- Data is processed across 30 countries but must remain decryptable — **multi-Region keys** replicate the same key material across regions
- **Automatic key rotation** for compliance
- **Grants** provide temporary, fine-grained permissions without editing the key policy

PrivateLink endpoint policies

```
{ "Effect": "Allow",
  "Action": ["bedrock:InvokeModel"],
  "Resource": "arn:...:model/claude-*" }
```

From this enterprise network: these models only, invoke only — no fine-tuning, no deletion. Security groups further limit traffic to specific CIDR ranges.

AWS WAF

Rule type	Protects against
Rate-based	Abuse from a single source. Especially important for GenAI — every request costs money, so abuse is a billing attack.
Geographic match	Regional access control
SQL injection	Malicious query parameters

AWS RAM — cross-account sharing

Large enterprises run many accounts across OUs. Resource shares with **permission sets** and **tag-based conditions** control which FM resources can be shared with which organisational units.

Logging and detection

Service	Provides
CloudTrail	API call audit — who, when (Task 1.6)
CloudWatch	Metrics, logs, alarms
Security Hub	Aggregated security findings across the account — detects unauthorised access attempts and policy violations

The layered check

User signs in	
-> [Federation]	identity from the enterprise IdP
-> [Cognito trigger]	enterprise roles injected into the JWT
-> [WAF]	abuse and rate checks
-> [PrivateLink]	network path + endpoint policy
-> [IAM]	AWS-level permission + conditions
-> [Verified Perms]	application-level: this resource?
-> [KMS]	data encrypted
-> [CloudTrail]	everything recorded

4. Cross-Environment AI Solutions

Some data cannot move to the cloud. So the model moves to the data instead.

The three edge options

Service	Where it runs	Why you'd choose it
AWS Outposts	Your own data centre	Compliance — data legally cannot leave the premises. Size compute and storage to on-premises data volume; use local caching to minimise data movement.
AWS Local Zones	Major metro areas	Distance — users are concentrated somewhere far from the region. Route tables direct specific traffic there.
AWS Wavelength	Inside the carrier's 5G network	Mobile — ultra-low latency, because traffic never traverses the internet. Distribute traffic between Wavelength Zones and the parent Region by workload.

Direct Connect

- Dedicated physical connection, not the internet; bandwidth sized to data transfer requirements
- BGP routing policies prioritise model inference traffic over batch processing** — inference is real-time, batch can wait, both on the same link

Transit Gateway — domain isolation

Dev environment -> its own route table
Test environment -> its own route table
Prod environment -> its own route table

- Route propagation settings** determine which environments can reach each other
- Black hole routes** deliberately drop traffic — e.g. ensuring no path exists from dev to prod

Network Firewall

Stateful rule groups inspect traffic between FM components and enterprise systems. **Domain filtering** restricts outbound connections to approved endpoints — so a compromised component cannot exfiltrate data.

Control Tower — enforcing data residency

"EU data must be processed only in EU regions"
"Non-compliant resources must not deploy at all"

Controls with automated compliance checks block non-compliant deployments *before* they happen. Recall the Task 1.1 problem where Global Accelerator had no native jurisdiction setting — Control Tower enforces this at organisation level instead.

Replication that respects boundaries

Technique	Effect
Filtering	Sensitive fields are never sent
Anonymization	Identity removed
Tokenization	Real values replaced by tokens — the actual value never crosses the boundary , and the model works on tokens

Finally, build monitoring and compliance verification with reporting and remediation — defining the rules is not enough, adherence must be proven.

5. CI/CD and the GenAI Gateway

CodePipeline — GenAI-specific stages

Source
-> Build
-> Model Evaluation <- GenAI-specific
-> Security Scanning <- GenAI-specific
-> Approval Gate
-> Deploy

Conventional software runs unit tests; here the pipeline must test **model behaviour** — accuracy, bias, prompt regression (Task 1.6). The approval gate before production echoes Tasks 1.2 and 1.6.

CodeBuild and CodeDeploy

- CodeBuild:** compute-optimized environments for model testing, and **caching of model artifacts and dependencies** — otherwise every build re-downloads tens of GB

- **CodeDeploy: healthy host thresholds** and **deployment timeouts** (LLM containers load slowly — see Task 2.2), with **rollback triggers driven by CloudWatch alarms**

CloudWatch composite alarms

```
IF    (latency > threshold)
AND  (confidence < threshold)
AND  (error_rate > threshold)
-> complex failure pattern detected
```

No single metric reveals the problem — high latency alone might just be load. Composite alarms combine metrics to detect patterns, and **anomaly detection** sets thresholds from historical performance rather than fixed numbers.

X-Ray sampling rules

```
normal requests  -> 5% sampled
slow requests    -> 100% sampled
error requests   -> 100% sampled
```

Detailed traces where they matter, low overhead elsewhere. **Custom subsegments** isolate critical stages — for example retrieval versus inference.

Service Catalog and CloudFormation custom resources

- **Service Catalog portfolios** with **launch constraints** enforce security baselines and tagging — teams self-serve, but only from approved patterns
- **CloudFormation custom resources** implement FM-specific provisioning logic, with **DependsOn** for correct creation order and **condition functions** for environment-specific configuration

The GenAI Gateway

Without it: every team calls Bedrock directly with its own credentials and code. Nobody knows total spend, guardrails are applied inconsistently, there is no unified audit trail, and every model change must be communicated to every team.

```
Team A --+
Team B ---+--> GenAI Gateway --> Bedrock / SageMaker
Team C --+
```

Function	How
Access control	Who may use which model
Rate limiting	Usage plans per team
Cost tracking	Consumption attributed per team
Guardrails	Applied centrally — no team can bypass them
Model routing	Cascading and fallback logic in one place
Caching	Shared across all consumers
Audit logging	Every invocation recorded
Abstraction	Swap the model without touching any team's code

```
API Gateway (usage plans, API keys, throttling)
-> Lambda (routing, adapters, validation)
-> Bedrock / SageMaker

alongside: Cognito/IAM, Verified Permissions,
           CloudWatch, X-Ray, CloudTrail, ElastiCache
```

This is the Task 1.2 abstraction layer at **enterprise scale**. Its real payoff is **centralised policy enforcement**: change a compliance rule in one place and it applies to every consumer, with automated remediation for common issues and defined escalation paths for exceptions.